Infrastructure Module Reference

Infrastructure Module Reference I

This section inventories every Layer-1 subdirectory returned by 23 `discover_infrastructure_modules(repo_root)`. File totals use 531 Python sources across infra + 7,385 infra tests guarding them. Documentation Duality = paired `README.md` + `AGENTS.md`; optional `SKILL.md` manifests feed `python -m infrastructure.skills`.

Module	Python Files	Has AGENTS.md	Has README.md	Key Exports
`autoresearch`	7			`build_autoresearch_plan`, readiness validation CLI
`benchmark`	3			Template harness scoring + comparative gates
`config`	0			Repository defaults + hardened templates
`core`	104			`get_logger`, `load_config`, `TemplateError`
`docker`	0			Containerisation scaffolding
`doctor`	14			Checkout diagnose/fix/undo repairs
`documentation`	11			`FigureManager`, `generate_glossary`
`llm`	54			Ollama helpers, sanitization, review + translation pipelines

Infrastructure Module Reference II

`logrotate.d`	0	Rotation snippets (documentation-first)
`methods`	5	`build_methods_orchestration_plan`, methods-stage contracts + validation
`orchestration`	8	`PipelineRunner`, entry point for `./run.sh`
`project`	21	`discover_projects`, workspace management
`prose`	8	Markdown readability + prose tooling
`publishing`	44	Zenodo, executable bundle, archival targets
`reference`	16	BibTeX models, parsers, converters
`rendering`	47	PDF/HTML/slide rendering, Pandoc filters
`reporting`	56	Coverage parsers, dashboards, executive artefacts
`scientific`	7	`check_numerical_stability`, `benchmark_function`
`search`	32	`infrastructure.search.literature` clients + cache
`sia`	9	Self-Improving-AI loop: task validation, harness, metric capture

Infrastructure Module Reference III

`skills`	6	`discover_skills`, SKILL manifest regeneration
`steganography`	10	Watermark overlays + hash manifests
`validation`	69	PDF + Markdown + integrity CLIs

Alphabetical summaries I

Below, `${module_*_python_file_count}` placeholders expand per subdirectory at render-time.

`infrastructure.autoresearch` (7 files)

Readiness planner, validation CLI, and report models for AutoResearch-style project promotion (`infrastructure/autoresearch/`).

`infrastructure.benchmark` (3 files)

Template harness scoring and comparative gate helpers exercised in CI smoke paths.

`infrastructure/config` (non-package subdirectory)

Repository-wide YAML templates and secure manifests (`.env.template`, hardened defaults referenced by Docker + CLI). `config/` carries no `__init__.py`, so it is a configuration subdirectory rather than an importable package.

Alphabetical summaries II

`infrastructure.core` (104 files)

Checkpointing, logging, pipeline YAML parsing, telemetry bridges, filesystem helpers, hardened exceptions. Everything else imports logging + error taxonomy from here first.

`infrastructure.doctor` (14 files)

Checkout diagnose/fix/undo repairs for broken local workspace states.

`infrastructure.docker` (0 files)

Pinned images / compose scaffolding for reproducible CI + remote builds.

`infrastructure.documentation` (11 files)

Figure registries plus glossary tooling feeding manuscript automation.

Alphabetical summaries III

`infrastructure.llm` (54 files)

Ollama integrations, sanitization adapters, templated reviewer flows. **Literature ingestion now lives primarily in `search/literature` + citation helpers in `reference/`.**

`infrastructure.methods` (5 files)

Deterministic methods-orchestration contracts (`MethodStage`, `MethodsOrchestrationPlan`, `MethodsIssue`): builds and validates an ordered methods plan for a research project so the manuscript's "Methods" track stays bound to executable stages.

`infrastructure.orchestration` (8 files)

`python -m infrastructure.orchestration` exposes interactive menus, subprocess wiring for thin shell wrappers (`run.sh`, `secure_run.sh`), and stubs used in CI for menu parsing tests.

Alphabetical summaries IV

`infrastructure.project` (21 files)

Canonical discovery (`discover_projects`) enforcing `src/` + `tests/`, slug validation, nested WIP namespaces.

`infrastructure.prose` (8 files)

Readability metrics + Markdown tooling for prose-centric manuscripts / CI gates.

`infrastructure.publishing` (44 files)

Metadata models, APA/BibTeX/MLA formatters, optional Zenodo clients.

`infrastructure.reference` (16 files)

Citation/BibTeX parsing + conversion utilities leveraged by manuscripts and retrieval scripts.

Alphabetical summaries V

`infrastructure.rendering` (47 files)

Pandoc shim, Unicode/XeLaTeX postprocessors, combined PDF/HTML/slide exporters.

`infrastructure.reporting` (56 files)

Parses pytest + coverage artefacts for dashboards; pairs with Stage 01 summaries and downstream executive exports.

`infrastructure.scientific` (7 files)

Stability probing, benchmarking hooks—consumed heavily by optimization exemplars (`template_code_project` scripts).

`infrastructure.search` (32 files)

`literature/` client stack (`client.py`, backends, caches) powering archive-only `template_search_project` literature workflows when copied locally from `projects/archive/`.

Alphabetical summaries VI

`infrastructure.sia` (9 files)

Generic Self-Improving-AI loop utilities: task-layout validation, execution harness, and metric capture reused by `template_sia` (fixture-replay by default).

`infrastructure.skills` (6 files)

Discovers `SKILL.md` frontmatter → `.cursor/skill_manifest.json`.

`infrastructure.steganography` (10 files)

Watermark overlays, hashing companions triggered by secure pipeline path.

`infrastructure.validation` (69 files)

Markdown + PDF + integrity CLIs underpinning Stage 04 diagnostics.

Alphabetical summaries VII

`infrastructure/logrotate.d` (0 files)

Operational templates for deployments (documentation-first; intentionally minimal Python footprint).

Documentation maturity: Coverage statements in Results pull from introspection—not hand-maintained denominators—so newly promoted modules automatically flow into manuscripts after `generate_manuscript_metrics.py`.

FAIR+RSE linkage: MCP-ready `SKILL.md` artefacts align with evaluator heuristics (executability + metadata richness) emphasized by FAIRsoft guidance [@garijo2024fairsoft].